AWS Multi-VPC Peering using Terraform

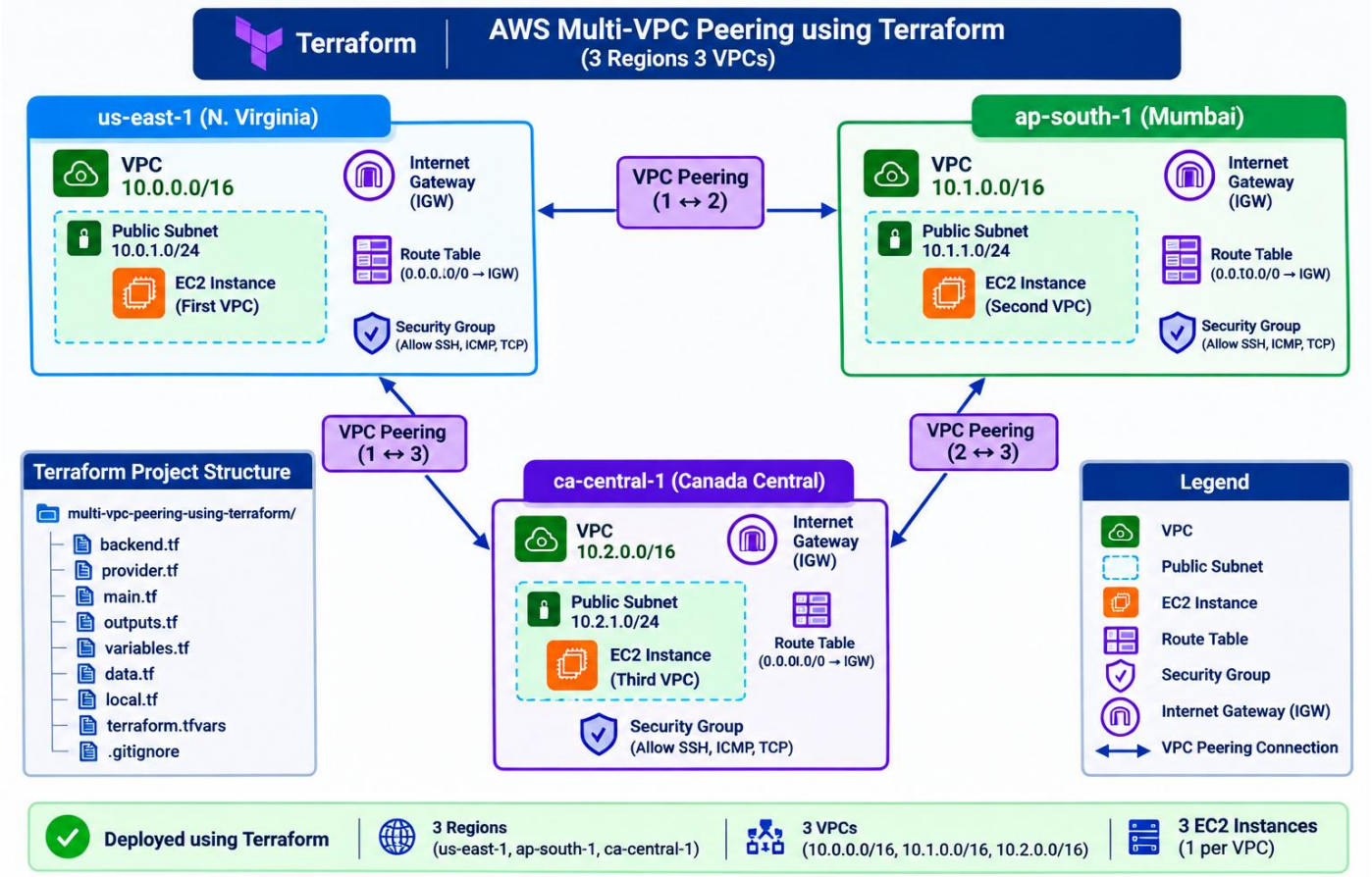
Prepared by: Pooja Daingade

Project Overview

The AWS Multi-VPC Peering using Terraform project demonstrates the implementation of a multi-VPC networking architecture on Amazon Web Services (AWS) using Terraform. In this project, three Virtual Private Clouds (VPCs) are created with separate CIDR blocks and connected through VPC Peering connections. Terraform is used to provision and configure the AWS infrastructure, including VPCs, subnets, route tables, Internet Gateways, Security Groups, VPC Peering connections, and EC2 instances.

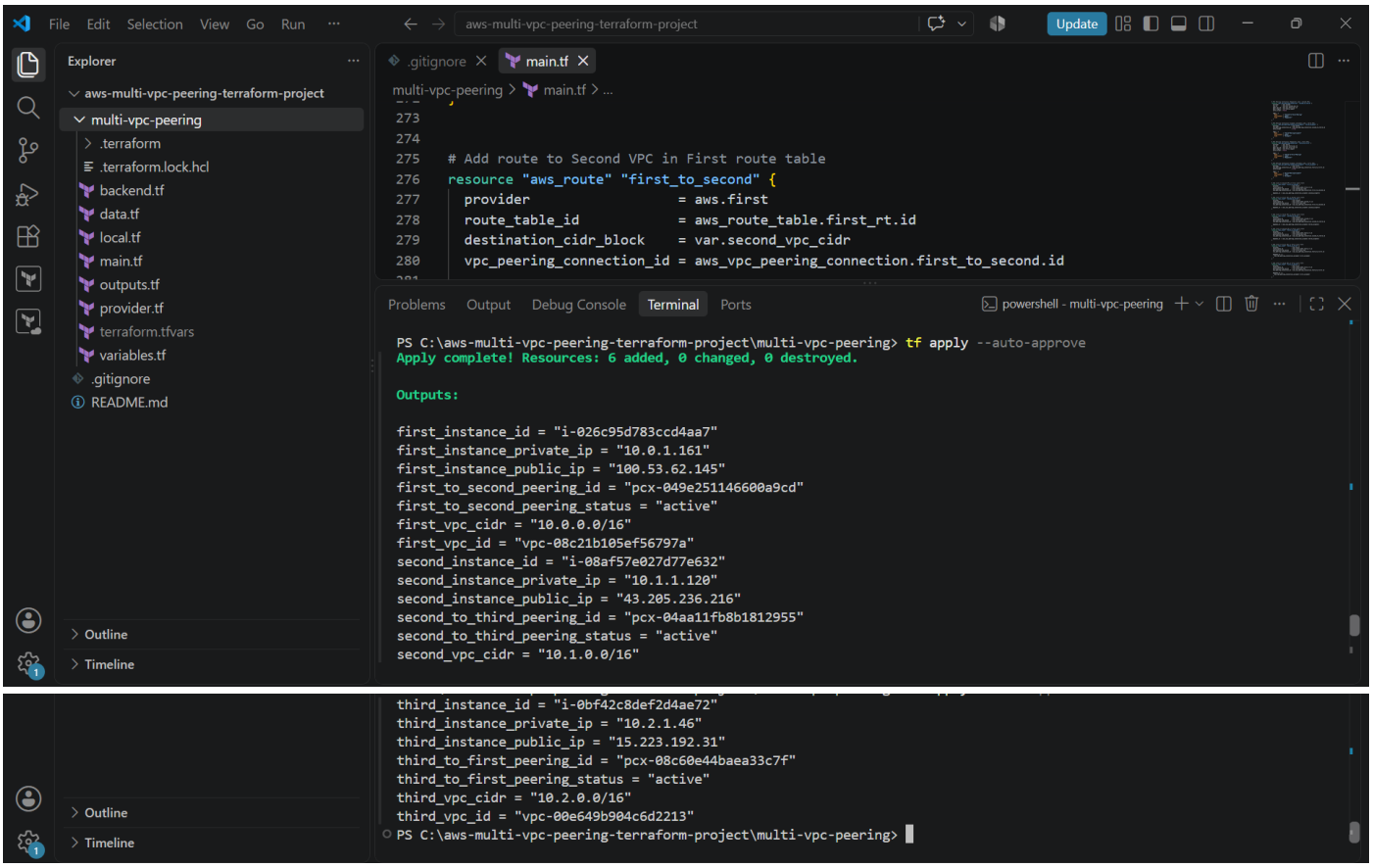
The architecture establishes a full-mesh topology between the three VPCs, allowing private communication between their resources through private IP addresses when the required routes and security rules are configured correctly. Connectivity is evaluated by performing ping tests between EC2 instances using their private IP addresses.

Architecture Diagram



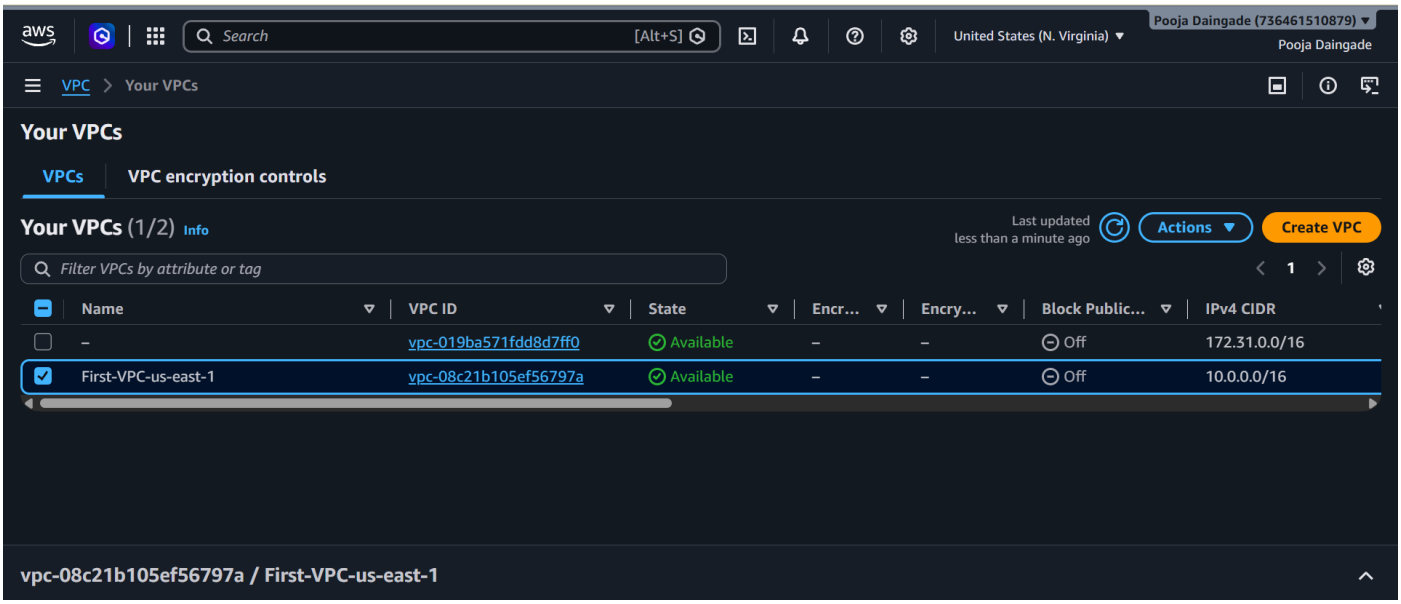
1. **Terraform Apply**

   Executed **terraform apply --auto-approve** to apply the Terraform configuration for the three-VPC architecture. Established VPC Peering connections between the first and second VPCs, second and third VPCs, and third and first VPCs. The Terraform apply operation completed successfully, and all three VPC Peering connections are in the **active** state.

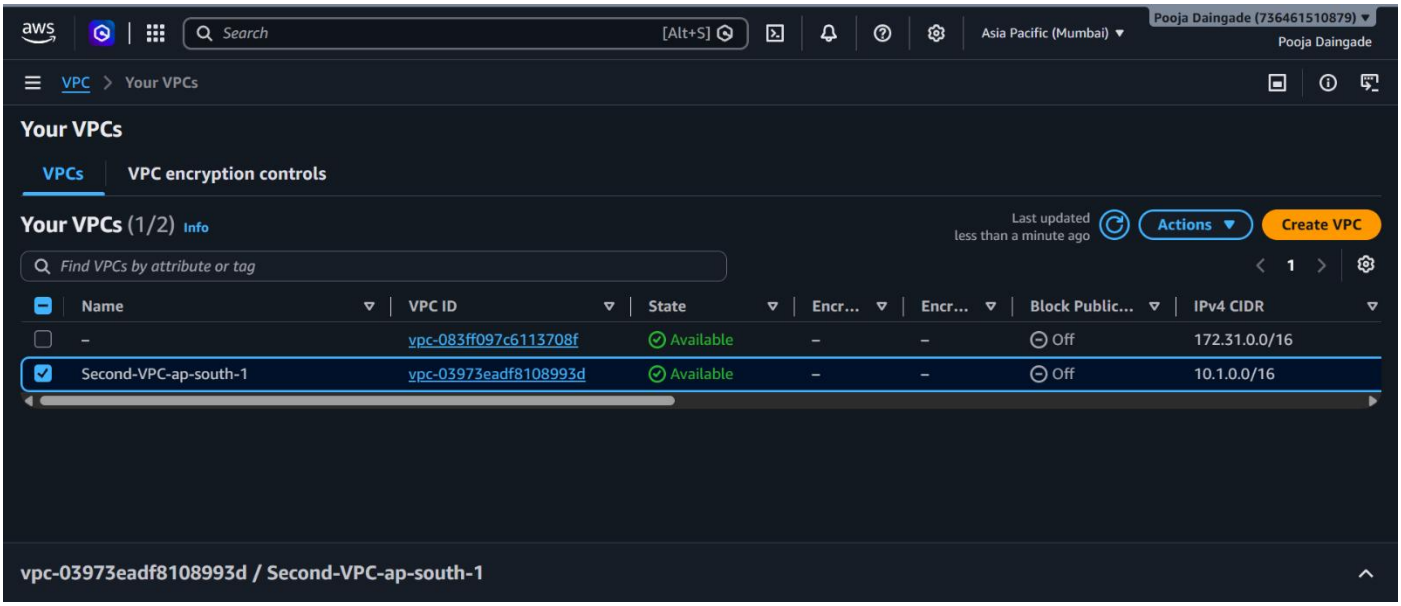


2. **VPC Creation**

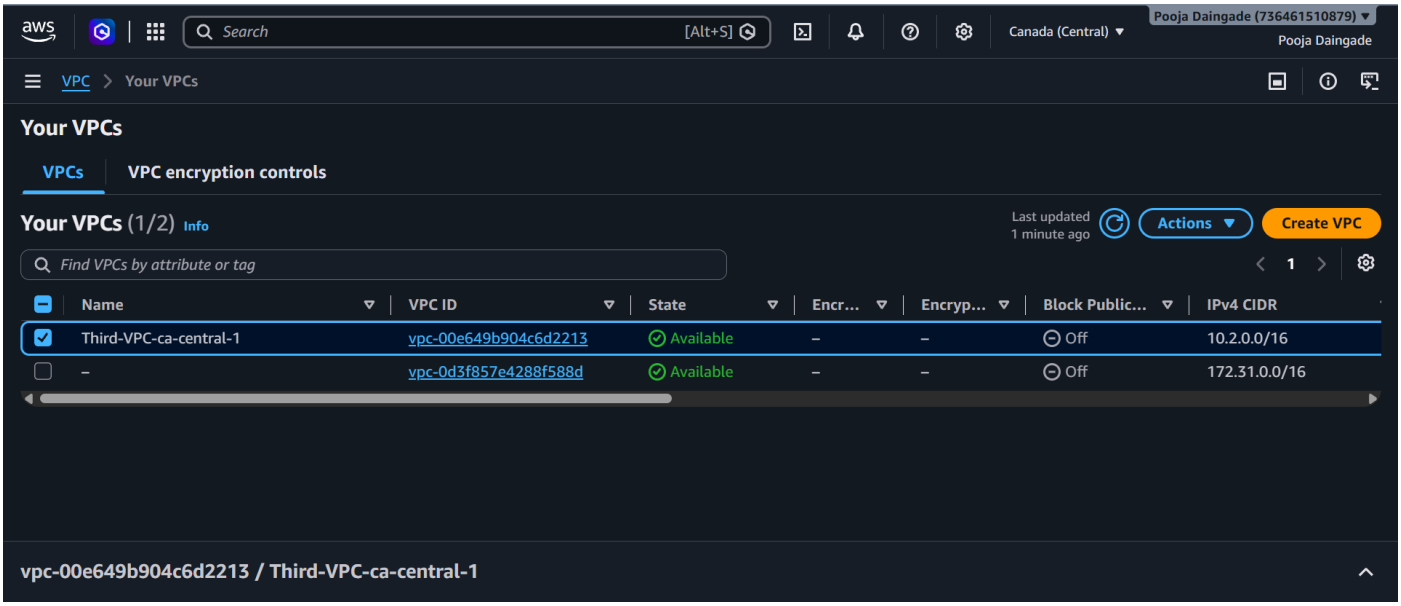
   - Created the first VPC in the us-east-1 (N. Virginia) region using Terraform with CIDR block 10.0.0.0/16



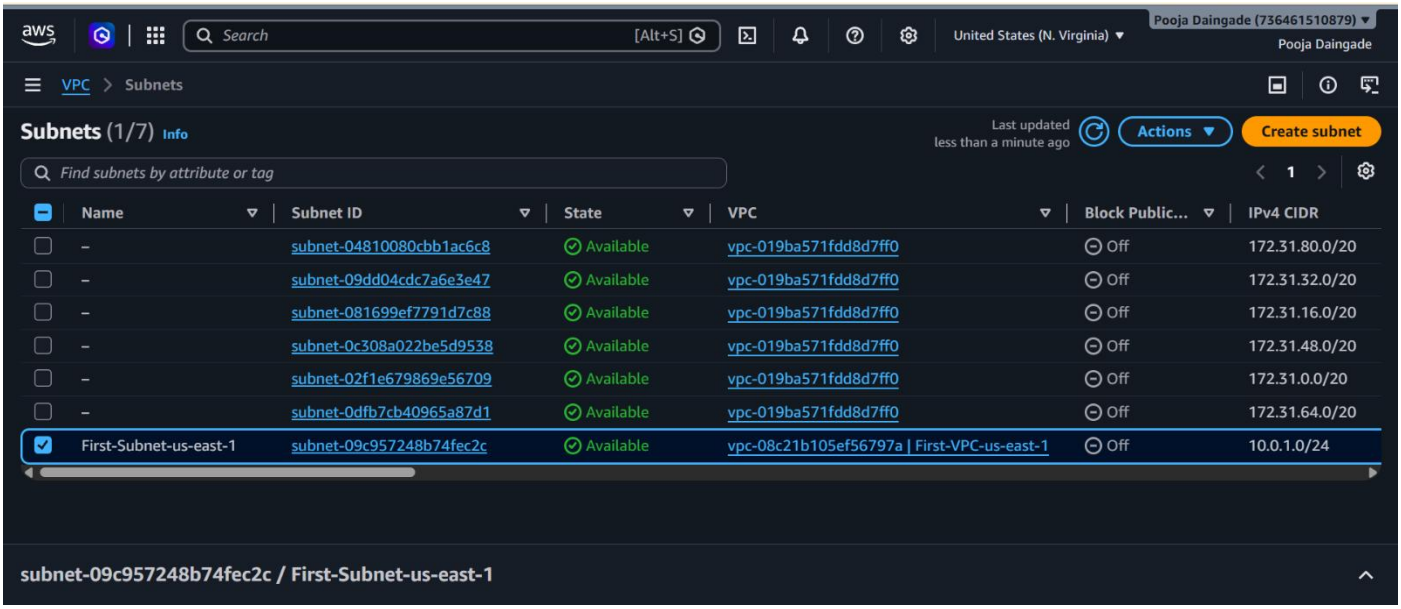
- Created the second VPC in the ap-south-1 (Mumbai) region using Terraform with CIDR block 10.1.0.0/16



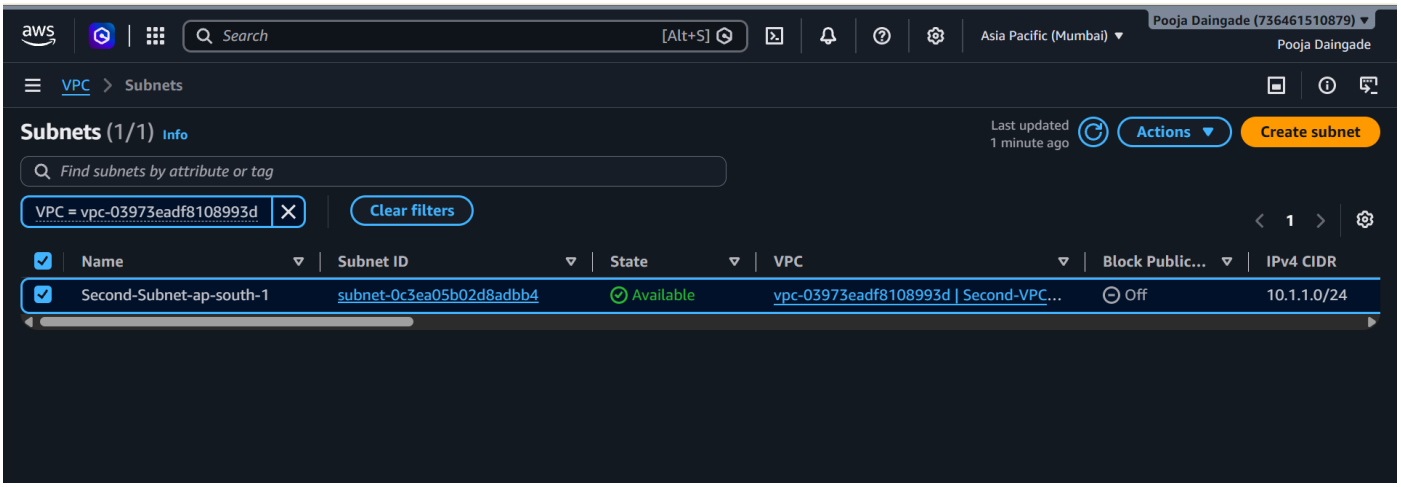
- Created the third VPC in the ca-central-1 (Canada Central) region using Terraform with CIDR block 10.2.0.0/16



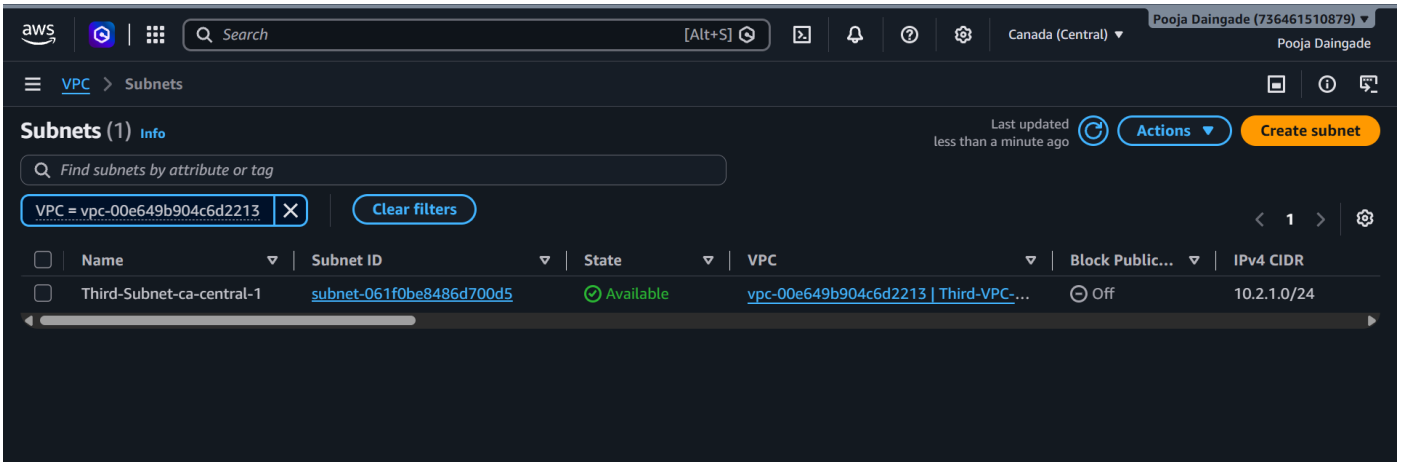
3. **Subnet Configuration**
   - Created a subnet in the first VPC (us-east-1) using Terraform with CIDR block 10.0.1.0/24



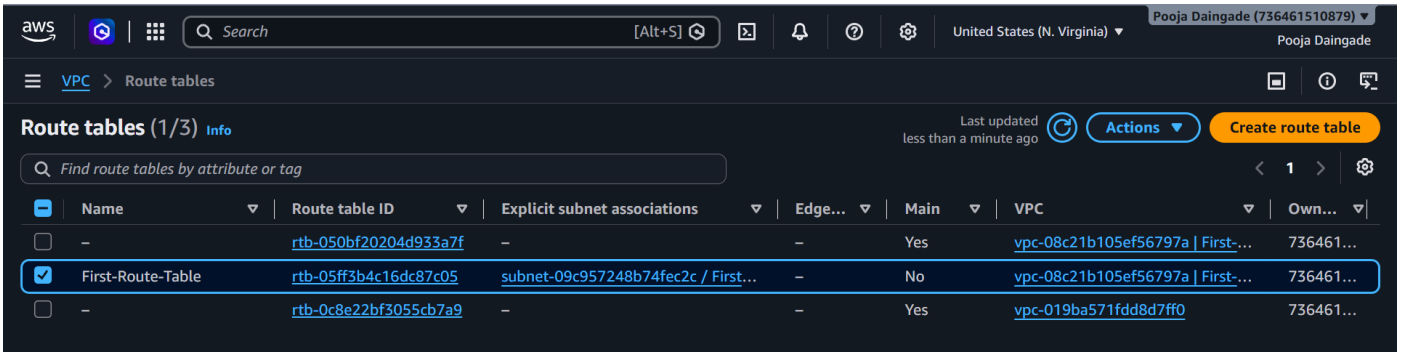
   - Created a subnet in the second VPC (ap-south-1) using Terraform with CIDR block 10.1.1.0/24



   - Created a subnet in the third VPC (ca-central-1) using Terraform with CIDR block 10.2.1.0/24.



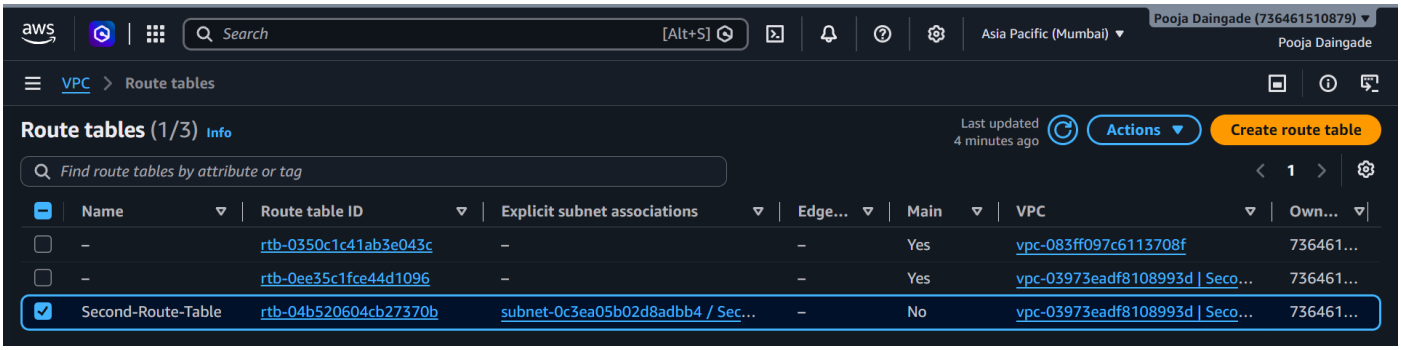
4. **Route Table Configuration**
   - Configured the First-Route-Table and associated it with the first subnet (10.0.1.0/24) in the First VPC.



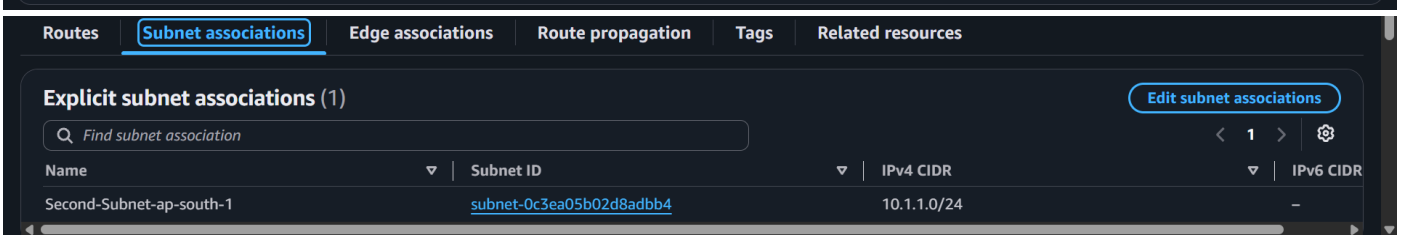




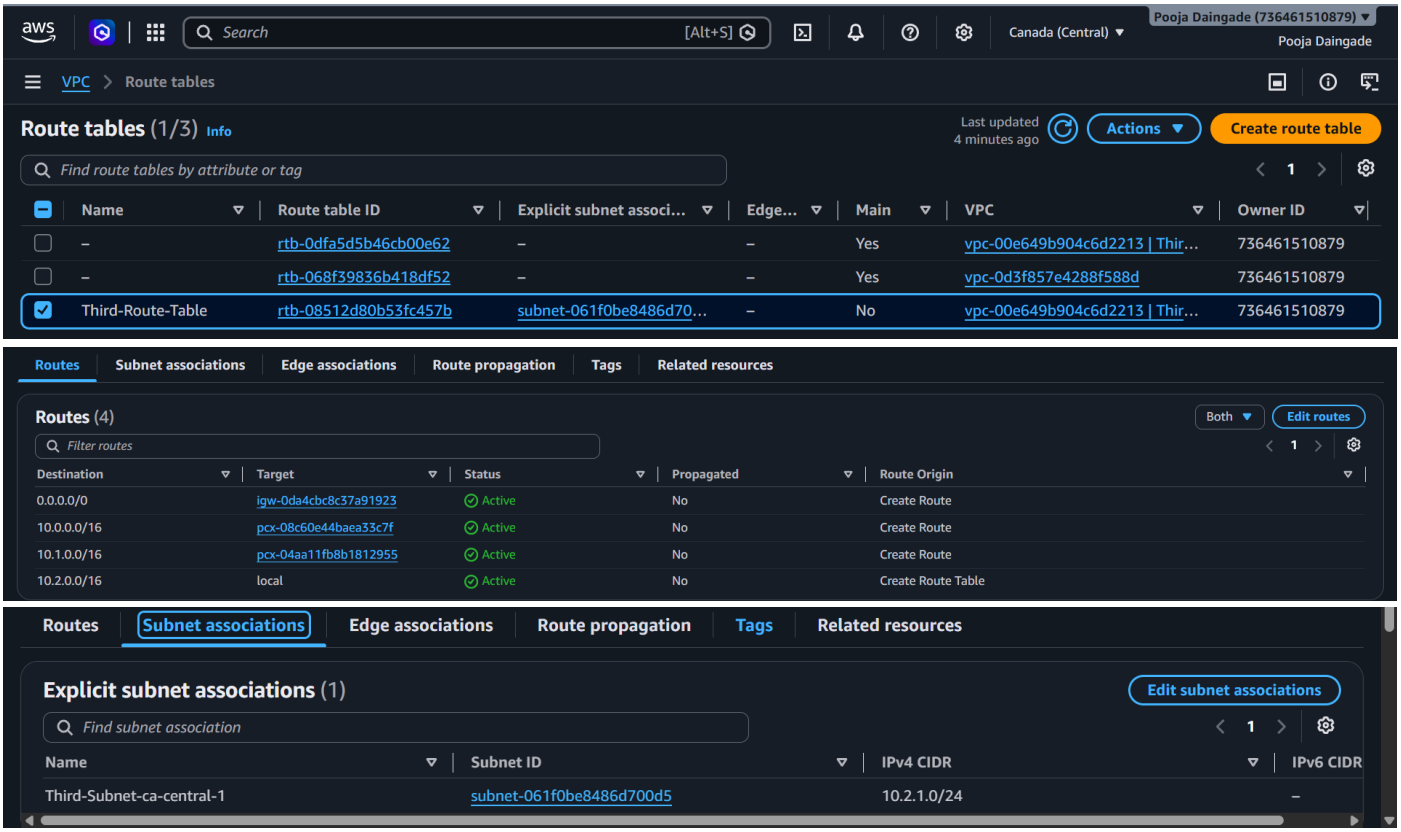
   - Configured the Second-Route-Table and associated it with the second subnet (10.1.1.0/24) in the Second VPC.





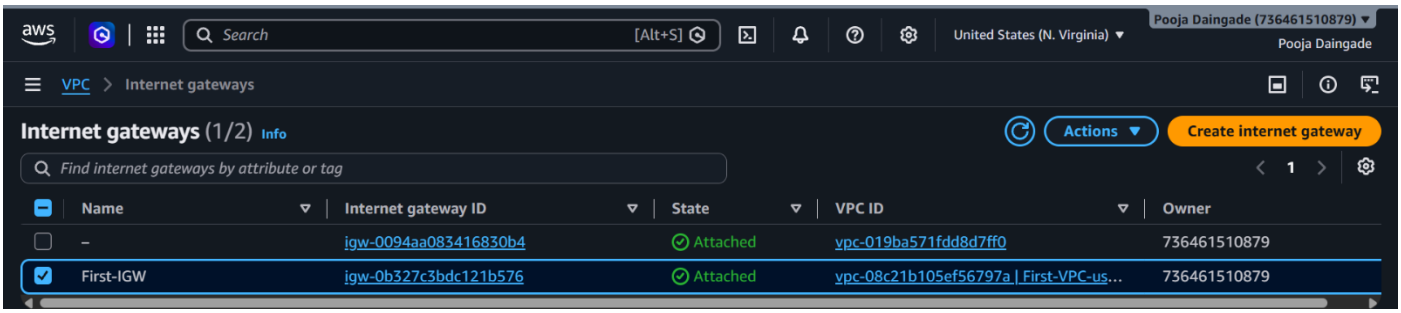


- Configured the Third-Route-Table and associated it with the third subnet (10.2.1.0/24) in the Third VPC.

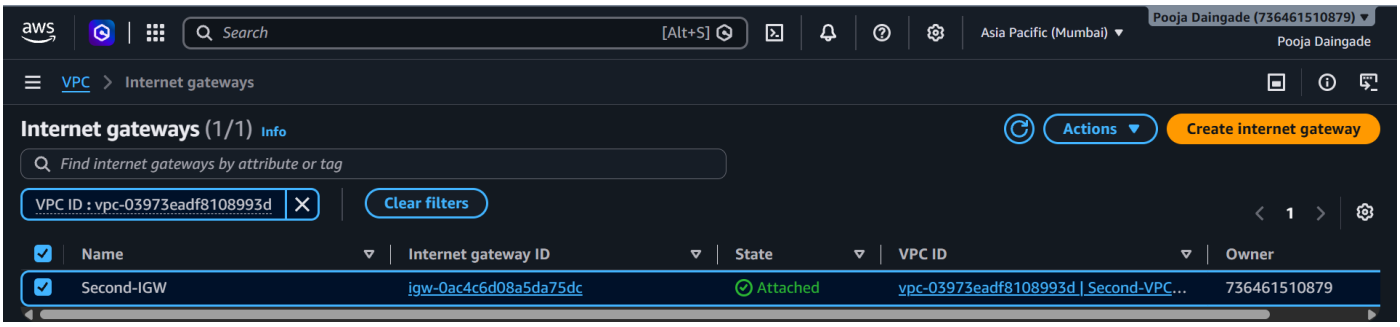


5. Internet Gateway Configuration

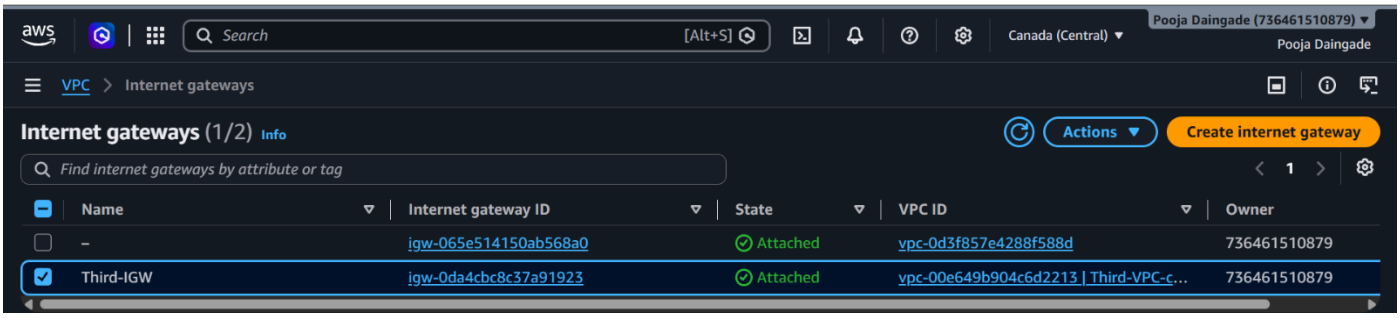
- Verified that First-IGW is attached to the first VPC in the us-east-1 region.



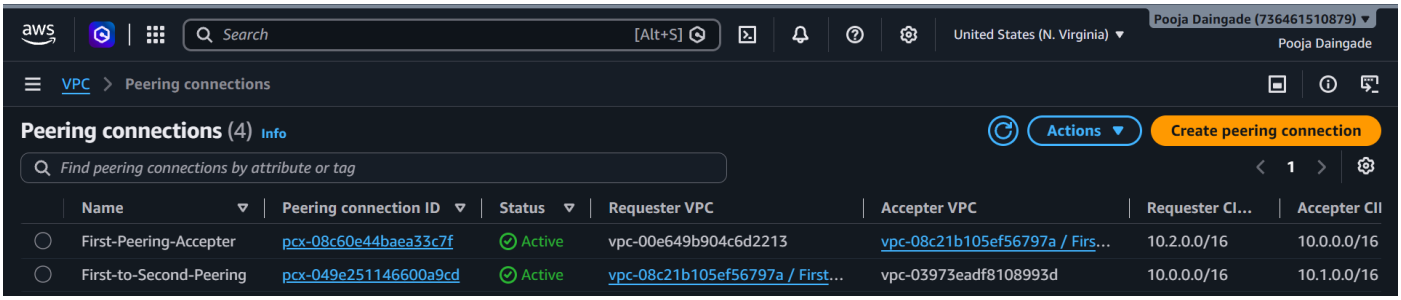
- Verified that Second-IGW is attached to the second VPC in the ap-south-1 region.



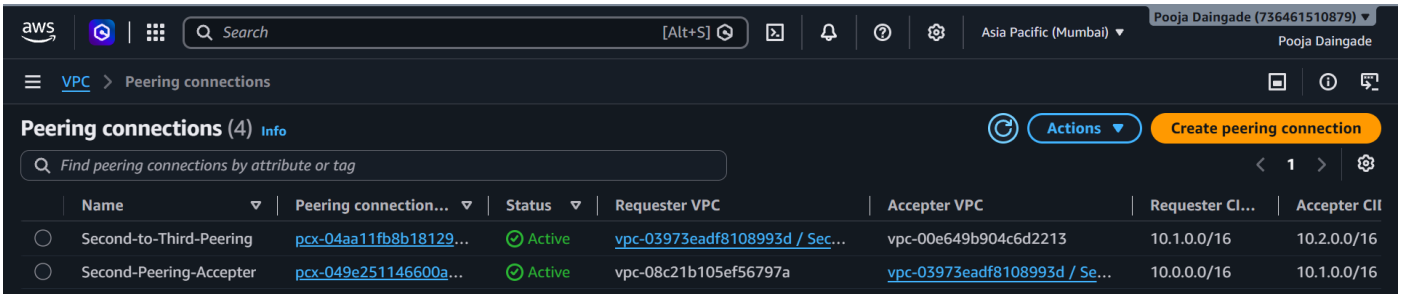
- Verified that Third-IGW is attached to the third VPC in the ca-central-1 region.



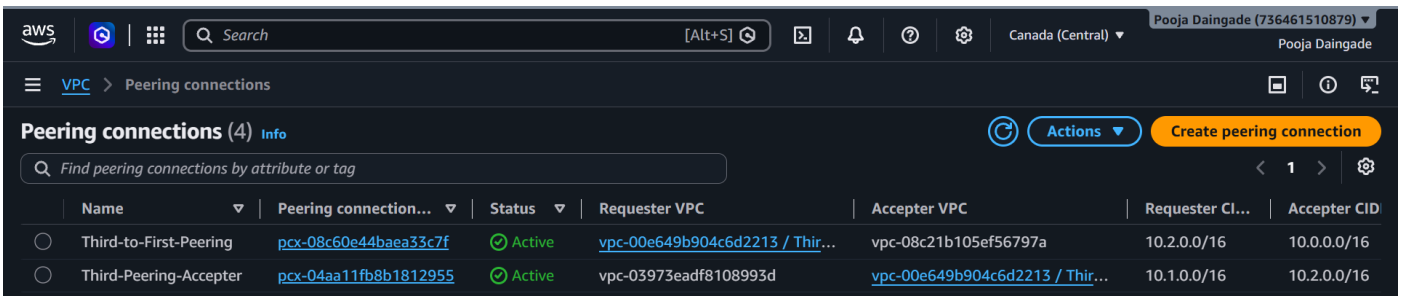
6. **VPC Peering Connections Verification in Three AWS Regions**
   - First-to-Second VPC Peering
     Verified the VPC Peering connection between the first VPC (10.0.0.0/16) and the second VPC (10.1.0.0/16). The connection status is **Active**.



   - Second-to-Third VPC Peering
     Verified the VPC Peering connection between the second VPC (10.1.0.0/16) and the third VPC (10.2.0.0/16). The connection status is **Active**.

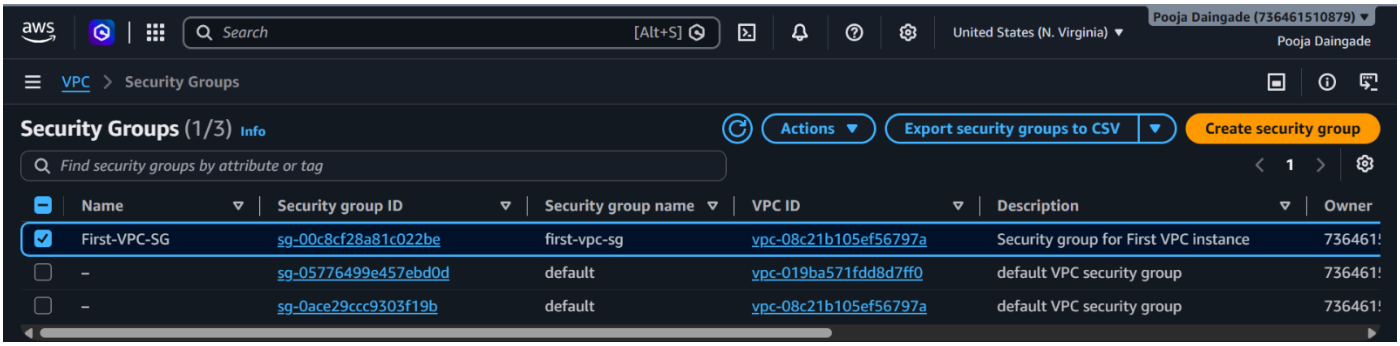


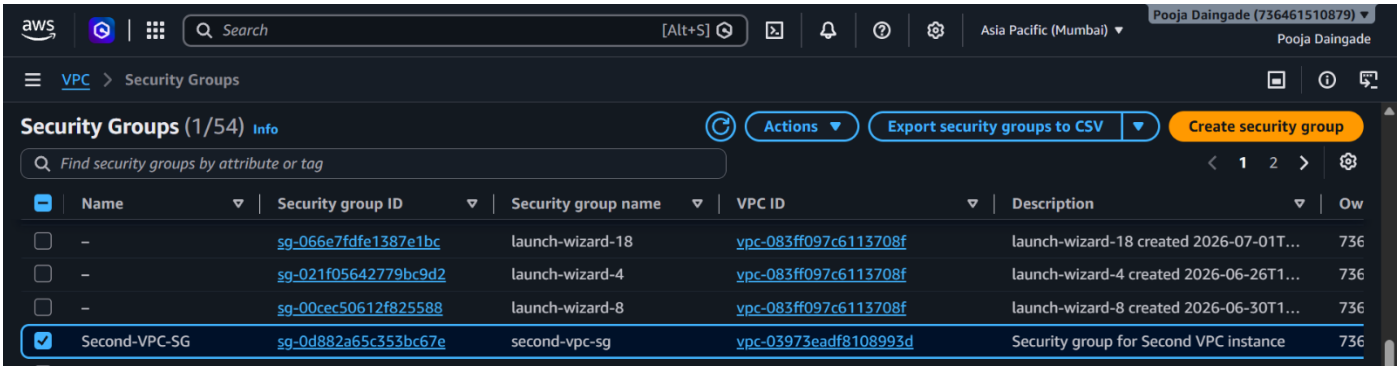
   - Third-to-First VPC Peering
     Verified the VPC Peering connection between the third VPC (10.2.0.0/16) and the first VPC (10.0.0.0/16). The connection status is **Active**.

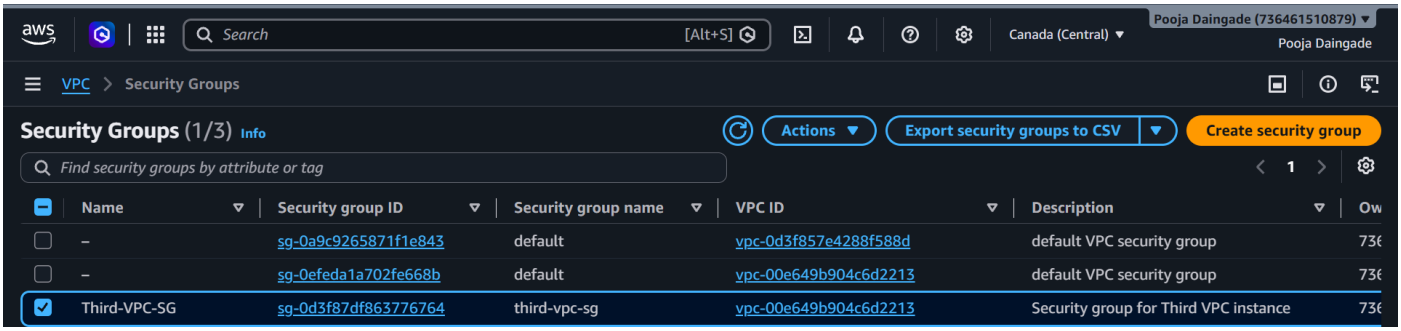


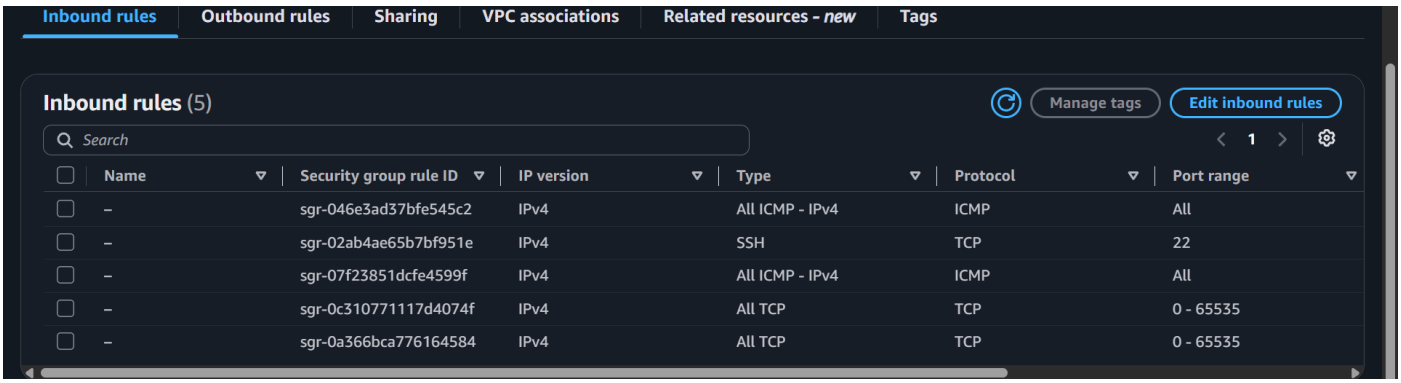
7. Security Groups

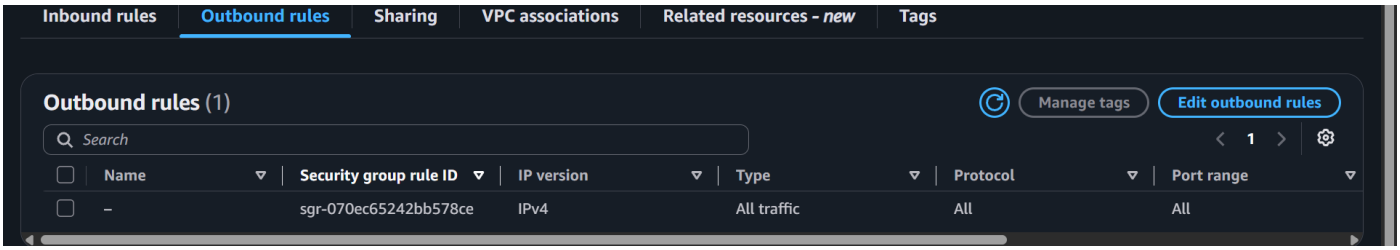
- Verified the First-VPC-SG, Second-VPC-SG, Third-VPC-SG security groups are created. Configured inbound rules to allow SSH, ICMP, and TCP traffic for remote VPC communication and connectivity testing. Outbound rules allowing all traffic.





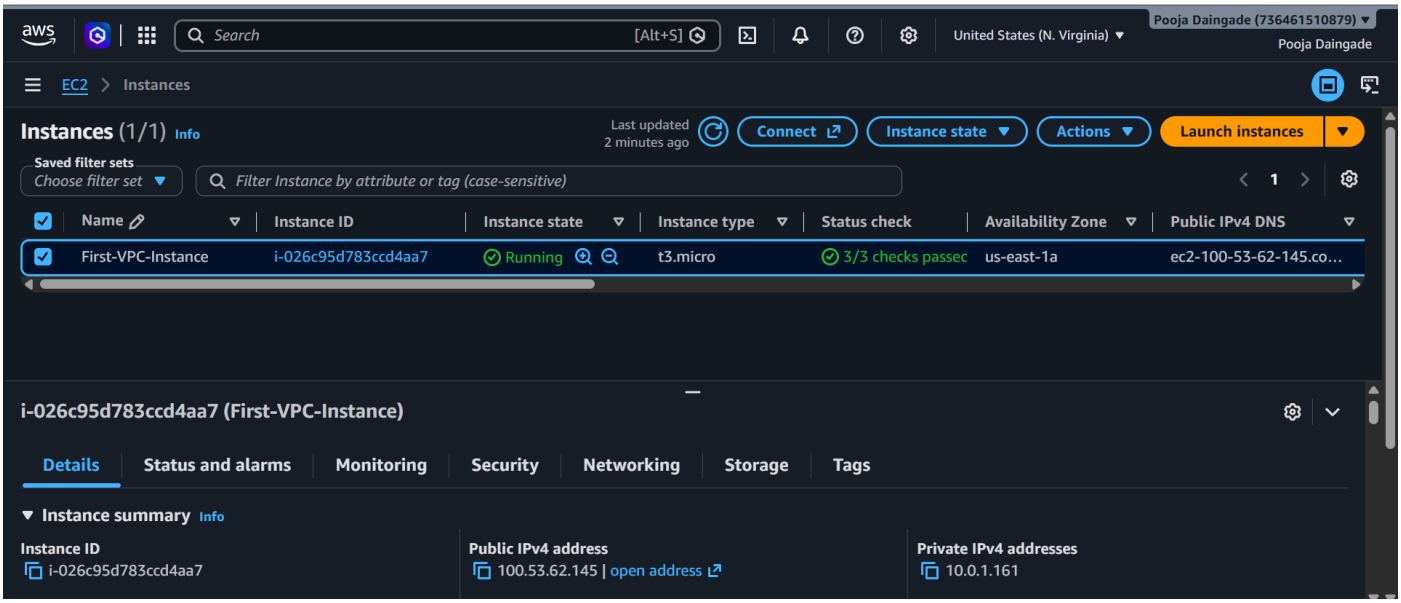




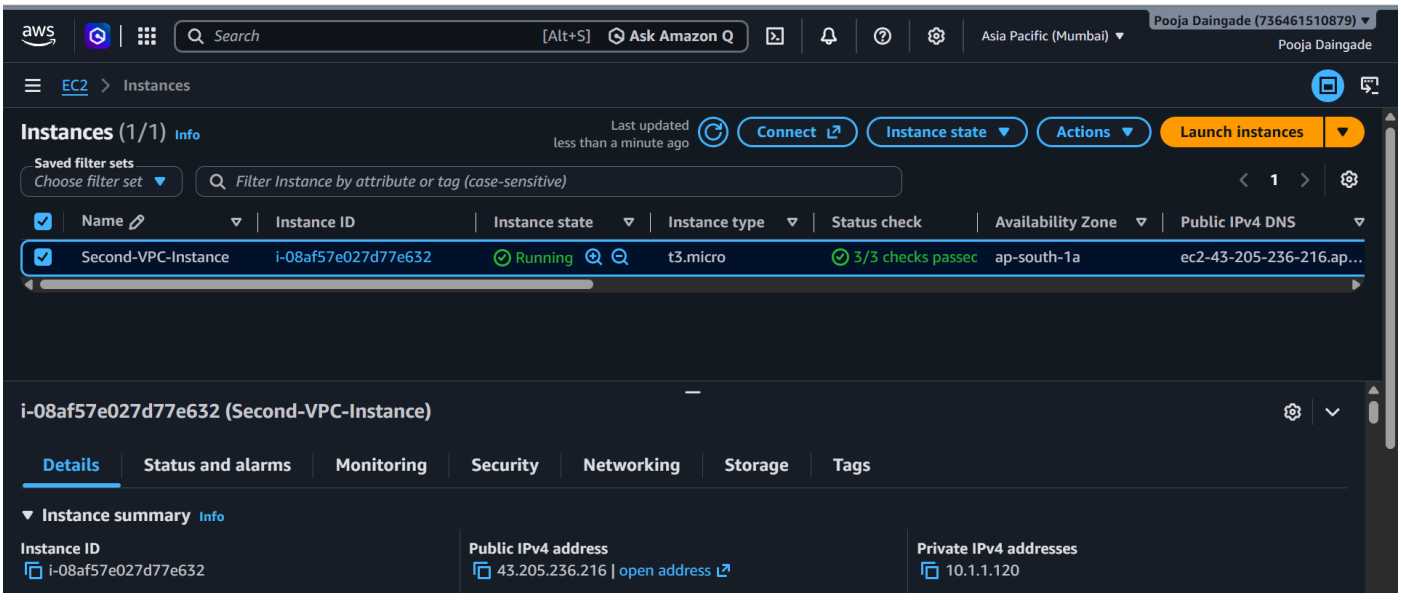


8. EC2 Instances Across Three AWS Regions

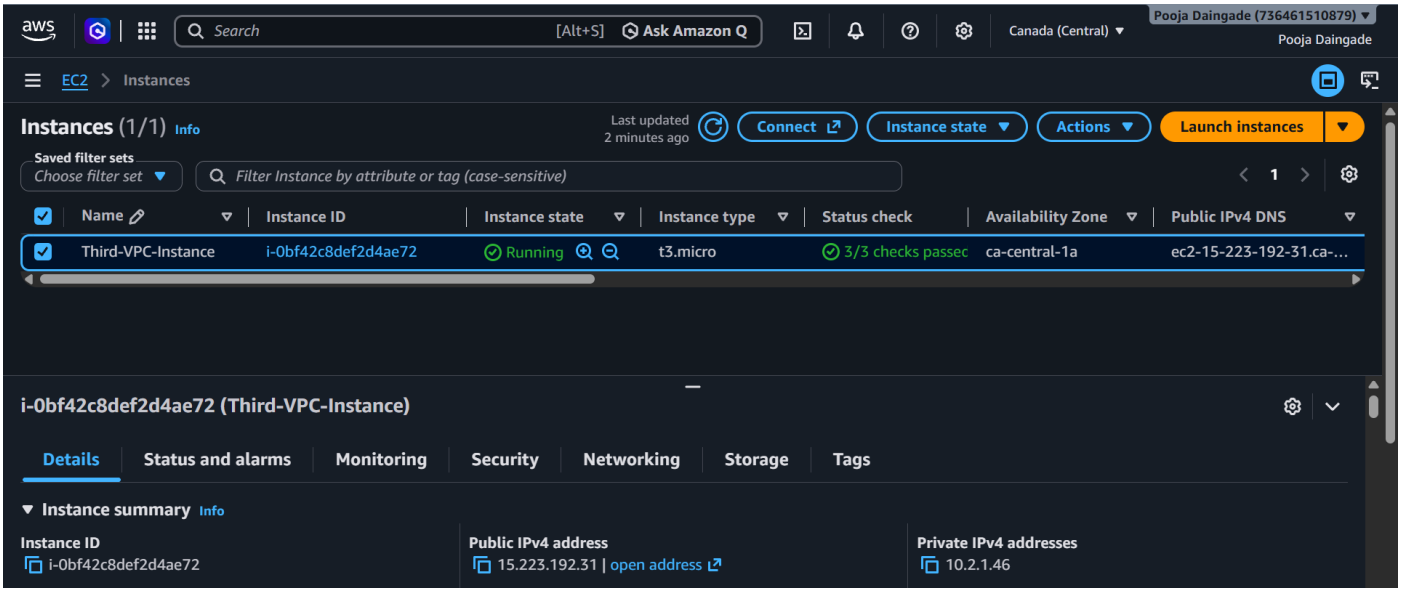
- Created an EC2 instance in the **region (us-east-1)** within the First VPC (10.0.0.0/16). The instance is deployed in subnet 10.0.1.0/24 with private IP 10.0.1.161 and instance type t3.micro.



- Created an EC2 instance in the **region (ap-south-1)** within the Second VPC (10.1.0.0/16). The instance is deployed in subnet 10.1.1.0/24 with private IP 10.1.1.120 and instance type t3.micro.

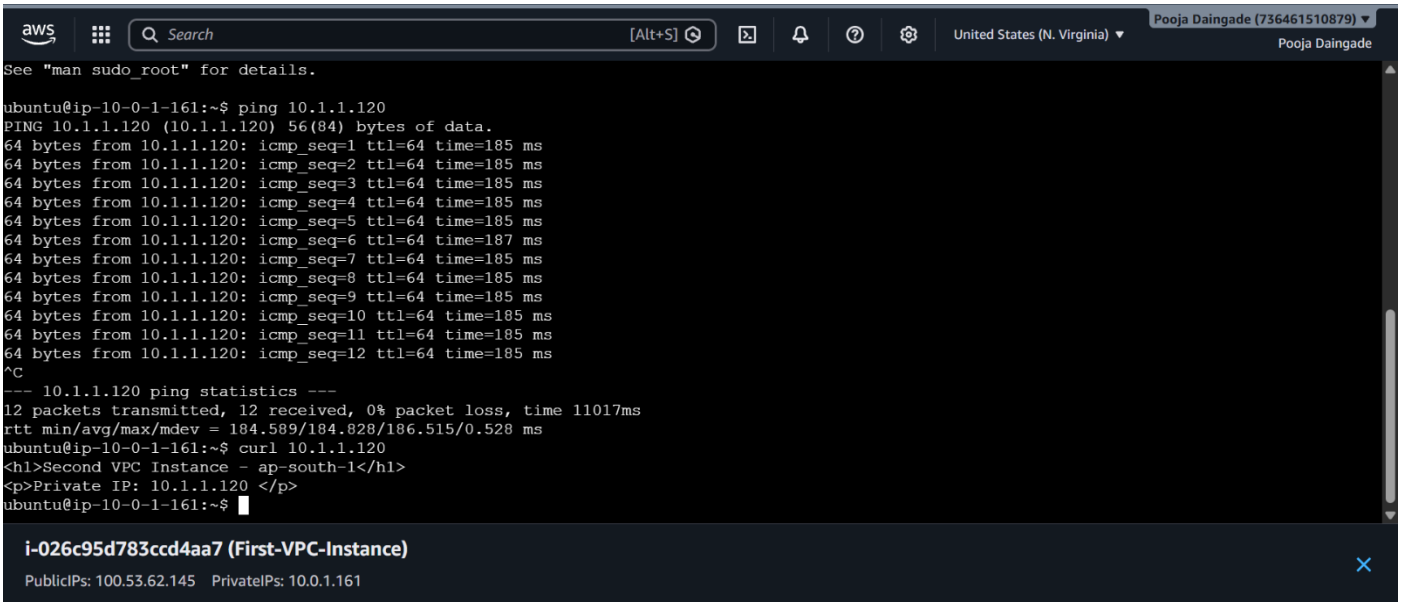


- Created an EC2 instance in the **region (ca-central-1)** within the Third VPC (10.2.0.0/16). The instance is deployed in subnet 10.2.1.0/24 with private IP 10.2.1.46 and instance type t3.micro.

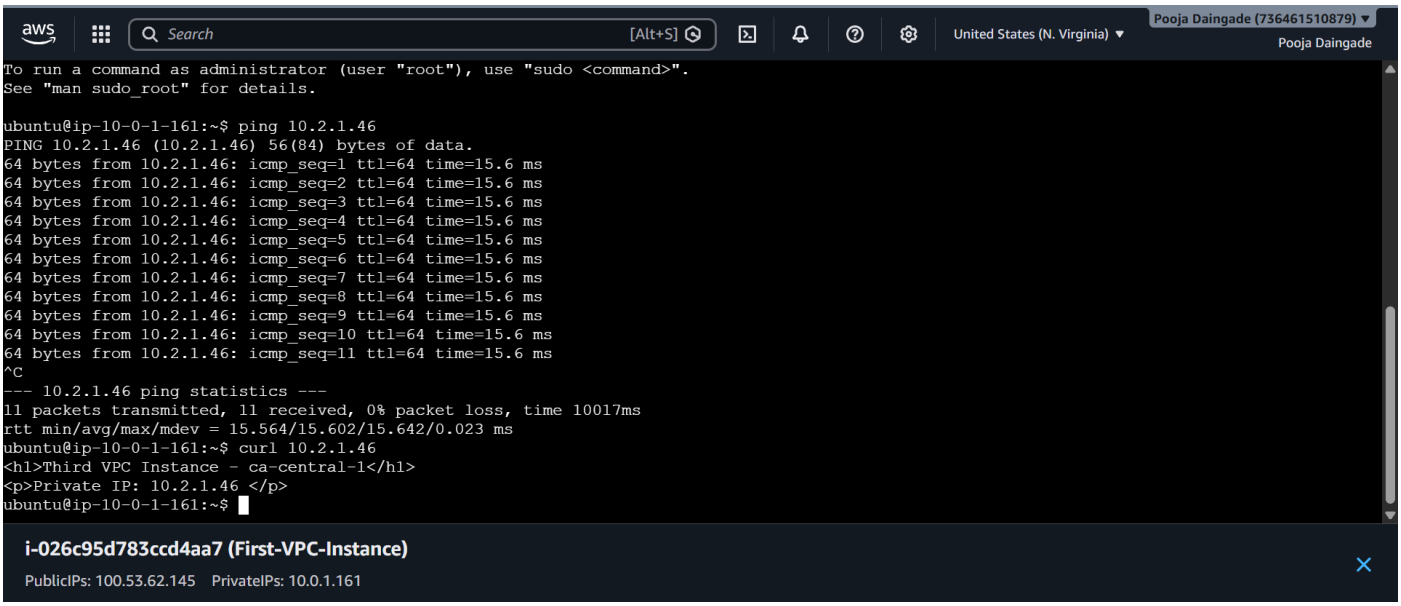


9. Connectivity Testing

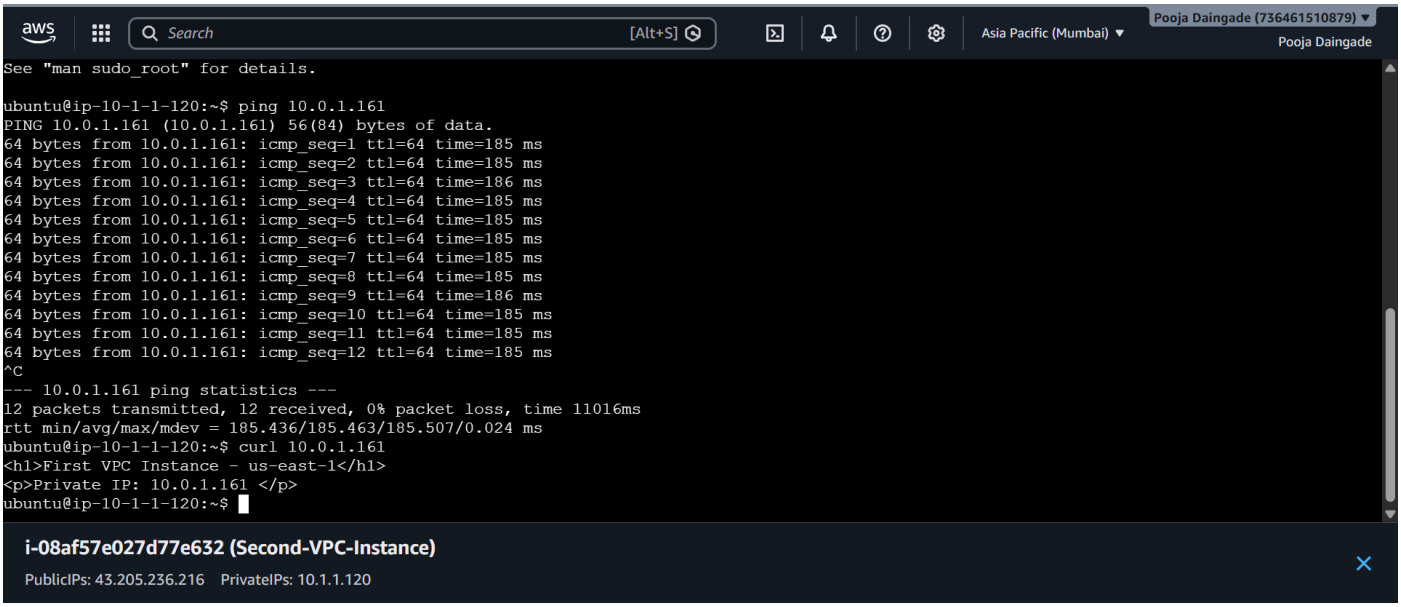
- **First VPC to Second VPC** - Executed the ping 10.1.1.120 command from the first EC2 instance to test connectivity with the second EC2 instance using its private IP address. The ping test was successful, confirming network connectivity between the two VPCs through the VPC Peering connection.
  The curl 10.1.1.120 command was also executed to test connectivity to the second EC2 instance.



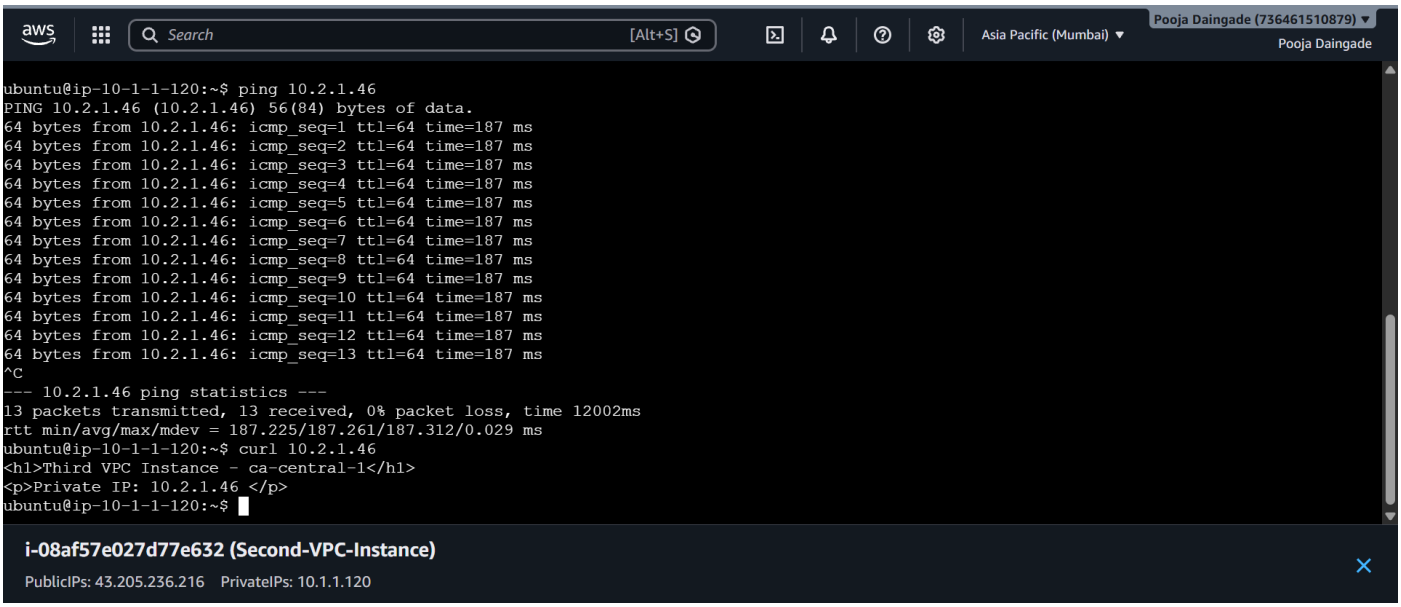
- **First VPC to Third VPC** - Executed the ping 10.2.1.46 command from the first EC2 instance to test connectivity with the third EC2 instance using its private IP address. The ping test was successful, confirming network connectivity between the two VPCs through the VPC Peering connection.
  The curl 10.2.1.46 command was also executed to test connectivity to the third EC2 instance.



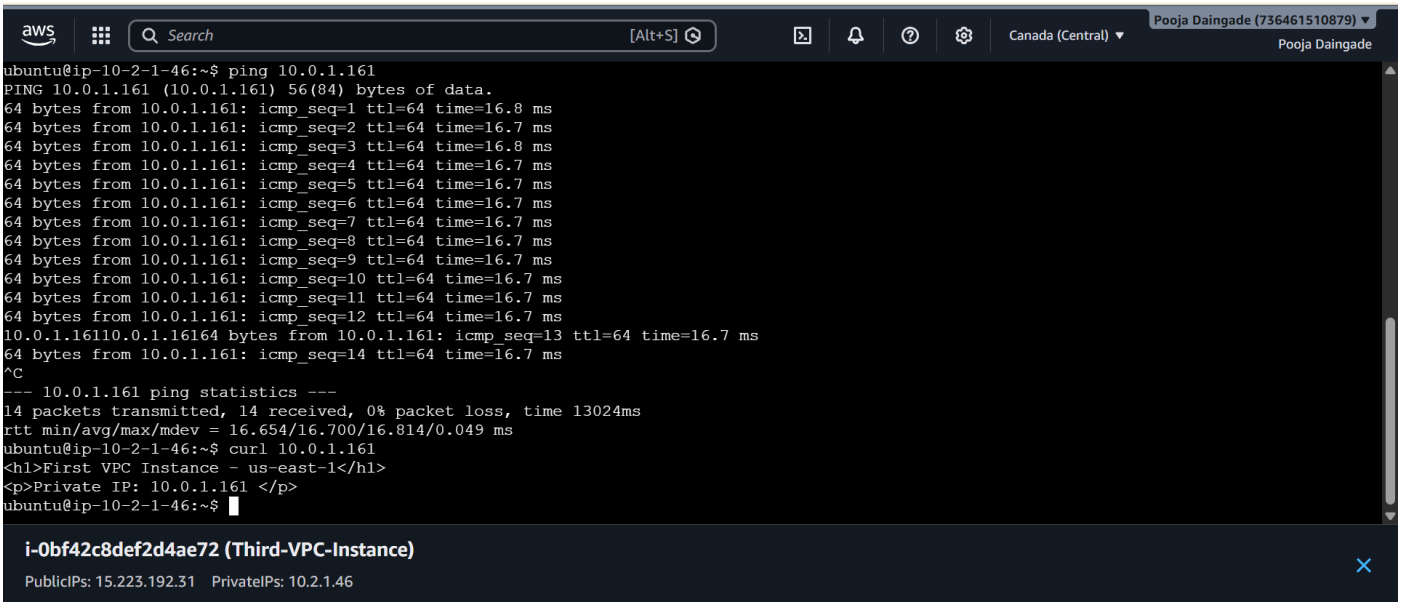
- **Second VPC to First VPC** - Executed the ping 10.0.1.161 command from the second EC2 instance to test connectivity with the first EC2 instance using its private IP address. The ping test was successful, confirming network connectivity between the two VPCs through the VPC Peering connection.
  The curl 10.0.1.161 command was also executed to test connectivity to the first EC2 instance.



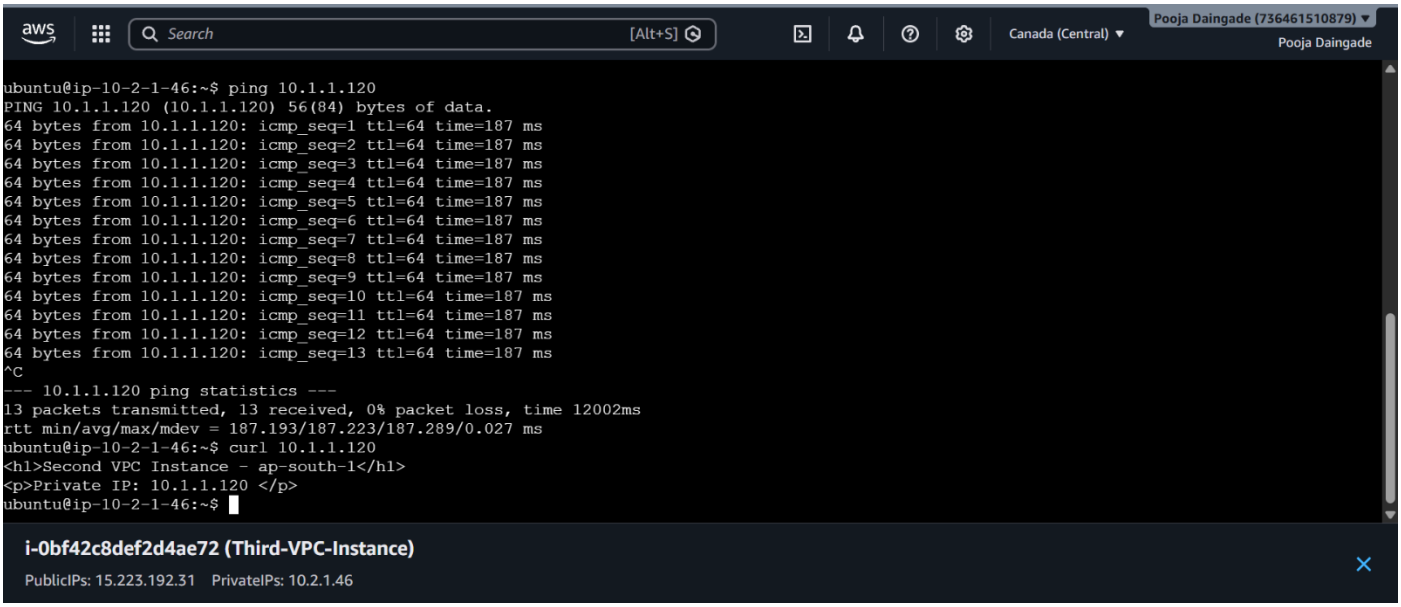
- **Second VPC to Third VPC** - Executed the ping 10.2.1.46 command from the second EC2 instance to test connectivity with the third EC2 instance using its private IP address. The ping test was successful, confirming network connectivity between the two VPCs through the VPC Peering connection.
  The curl 10.2.1.46 command was also executed to test connectivity to the third EC2 instance.



- **Third VPC to First VPC** - Executed the ping 10.0.1.161 command from the third EC2 instance to test connectivity with the first EC2 instance using its private IP address. The ping test was successful, confirming network connectivity between the two VPCs through the VPC Peering connection.
  The curl 10.0.1.161 command was also executed to test connectivity to the first EC2 instance.



- **Third VPC to Second VPC** - Executed the ping 10.1.1.120 command from the third EC2 instance to test connectivity with the second EC2 instance using its private IP address. The ping test was successful, confirming network connectivity between the two VPCs through the VPC Peering connection.
  The curl 10.1.1.120 command was also executed to test connectivity to the second EC2 instance.



Conclusion

Successfully implemented **AWS Multi-VPC Peering using Terraform** by creating three VPCs across different AWS regions and establishing full-mesh VPC Peering connections. Configured subnets, route tables, Internet Gateways, security groups, and EC2 instances using Terraform. Verified connectivity between EC2 instances using private IP addresses, demonstrating communication between the VPCs.